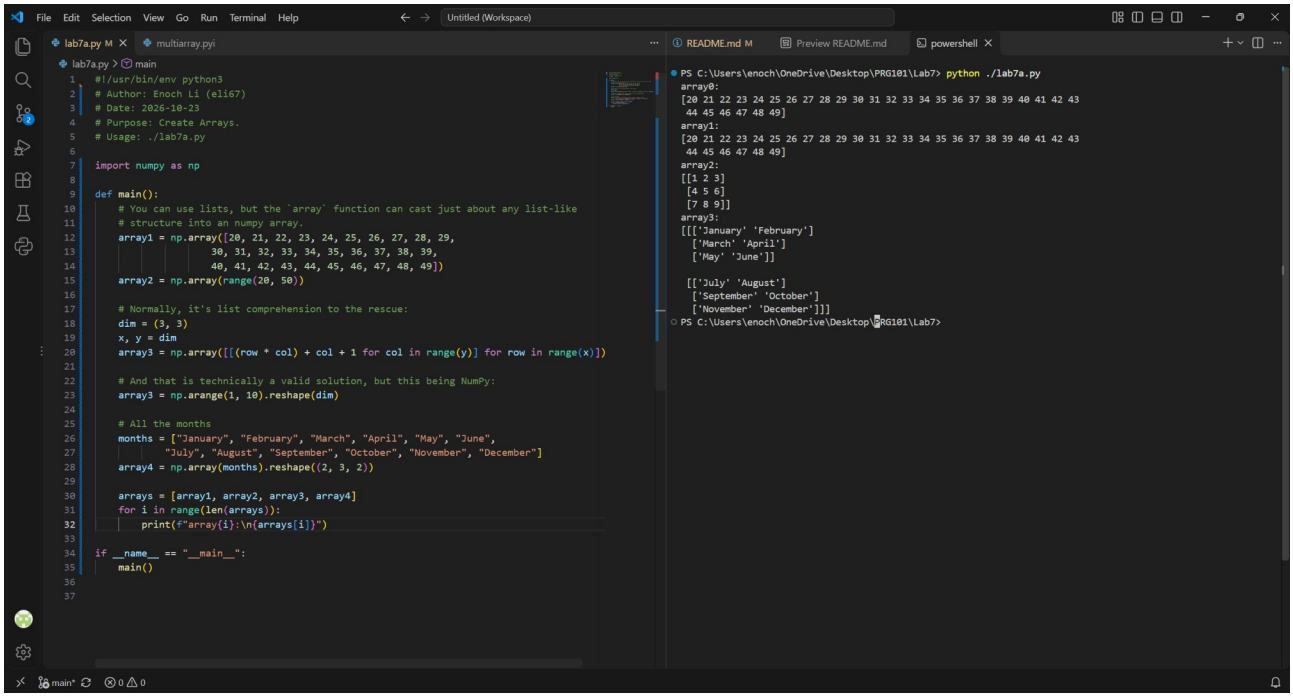
Lab 7

It's NumPy time!

Lab7a.py

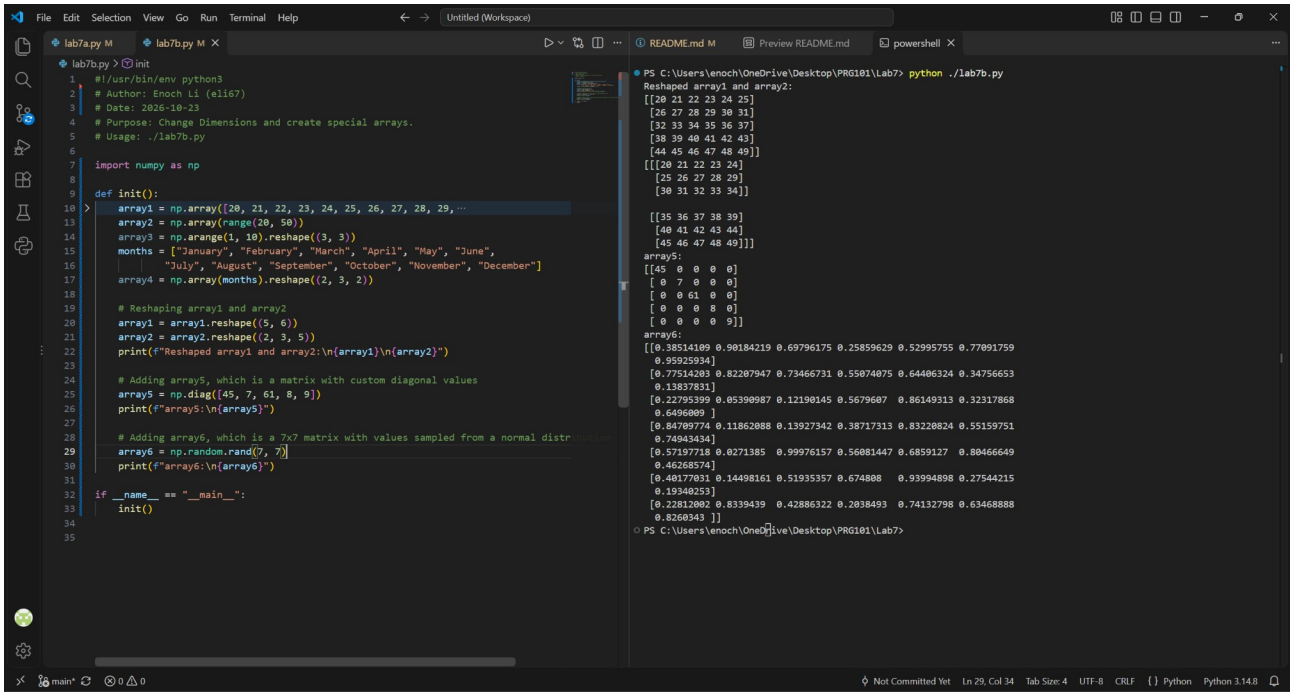
Because I'm doing these labs ahead of the lectures, I really do hope I'm interpreting all these instructions correctly.



Because NumPy is written in C, its operations are considerably faster than native Python, and is thus used to do a variety of things like mathematical computing and machine learning, which I suspect is the reason why we're even learning this.

Lab7b.py

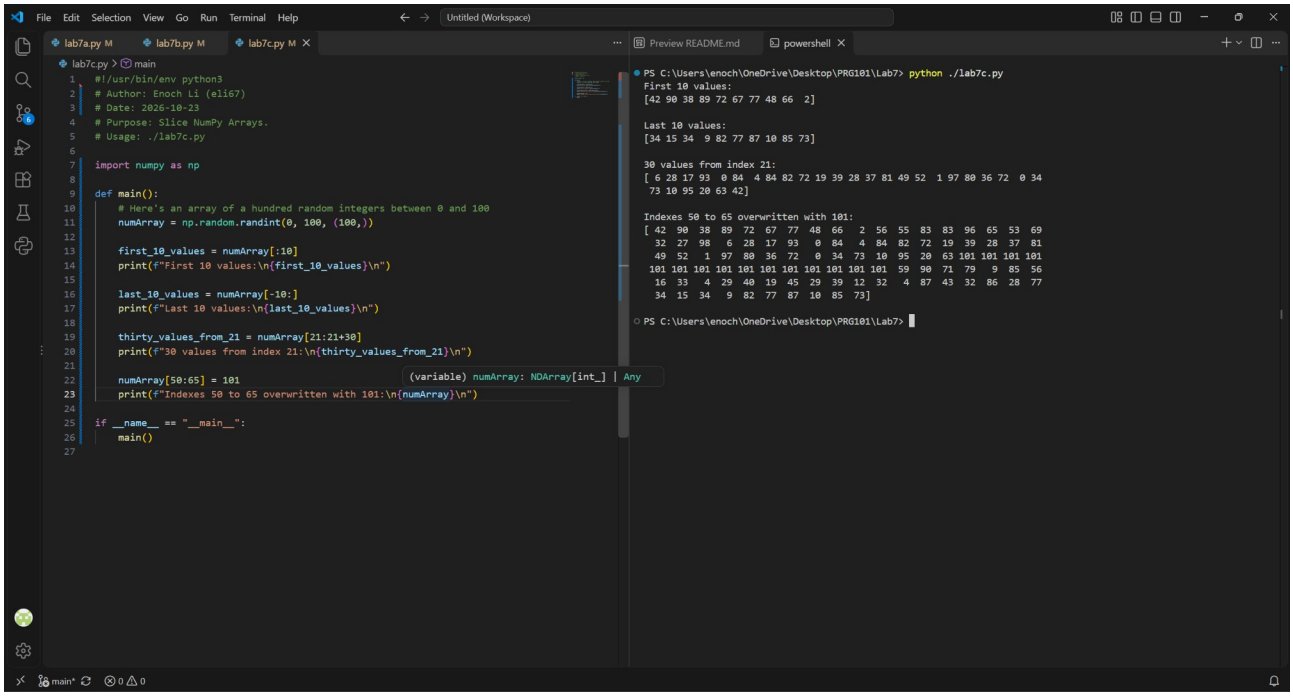
Yes, NumPy has a lot of options for manipulating arrays.



Only the relevant parts the code were pasted. The rest of it was discarded. Also, given the size of these outputs, the rest of the lab may have to be presented like this.

Lab7c.py

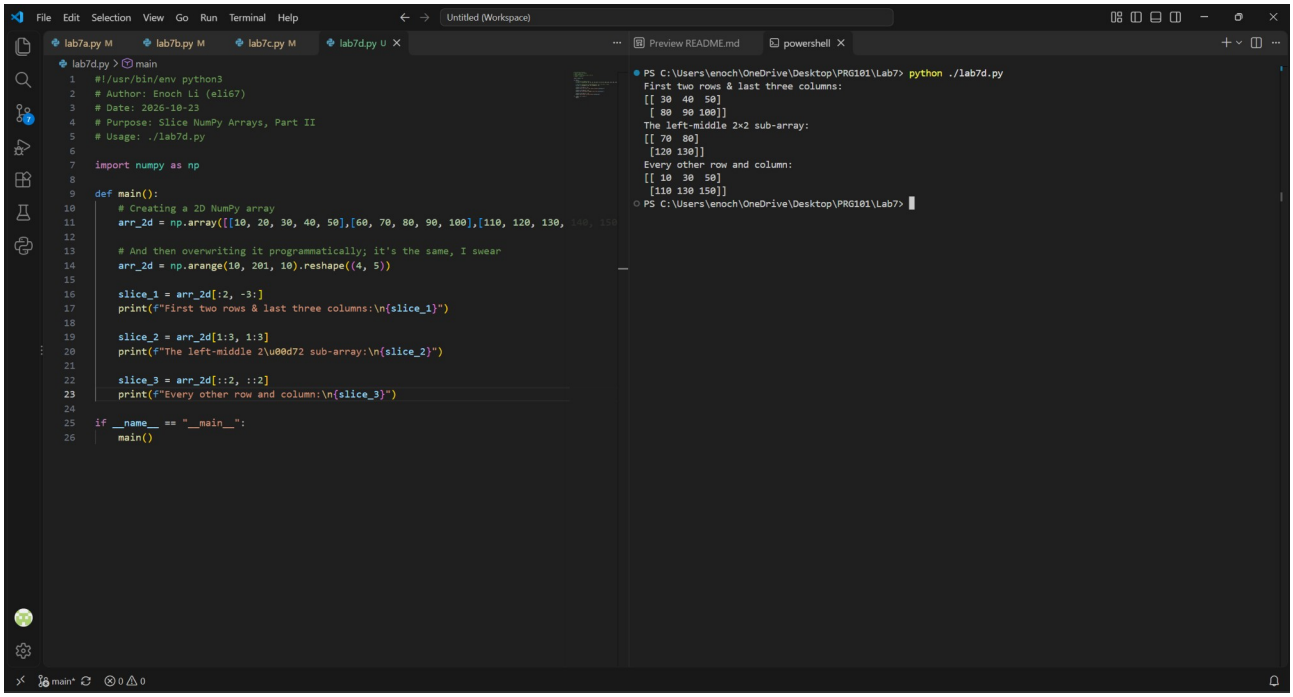
You know, you can make your own data structures indexable by array like this by inheriting from classes like `UserList`.



I would know because I've made a CasefoldedList class before.

Lab7d.py

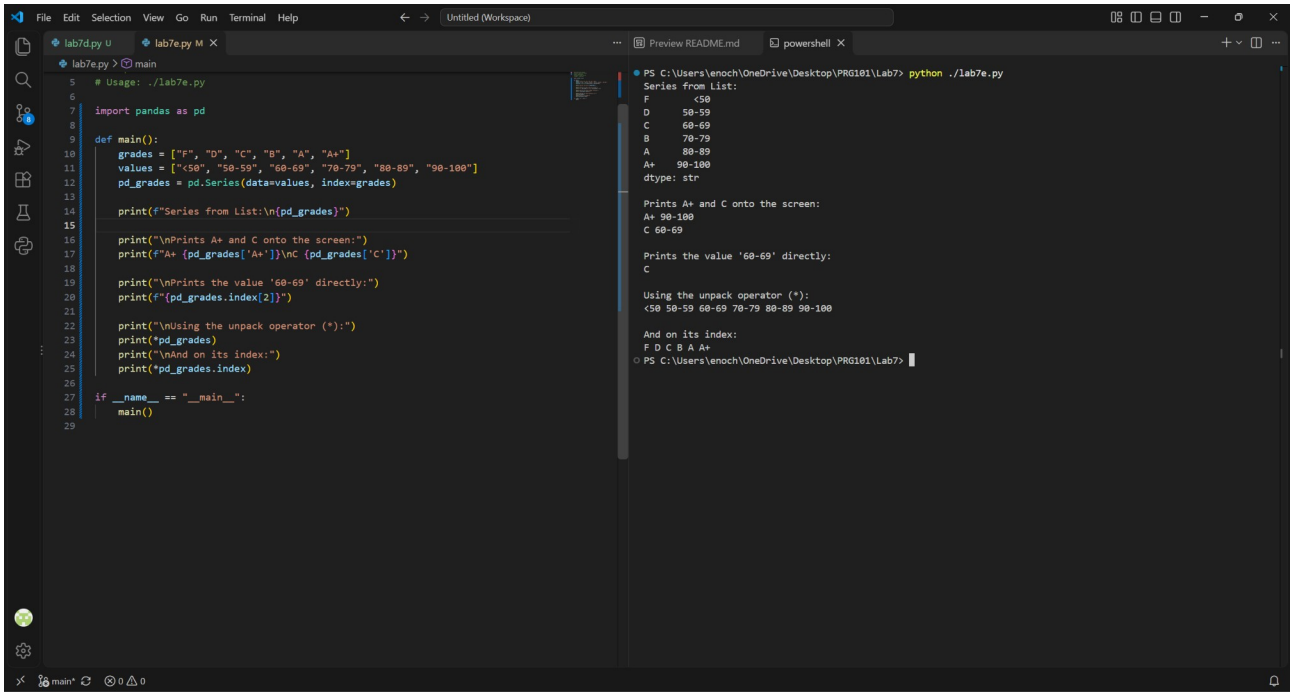
You can't tell me to copy an array that's so clearly the numbers from 10 to 200 in steps of ten in the shape of a (4, 5) array without expecting me to optimise it.



A slice has three argument: `start`, `stop`, and `step`. The third is rarely used, but all that means here is that because the first two arguments are empty, the slice will go through the entire array two steps at a time, and thus, return every other row and column.

Lab7e.py

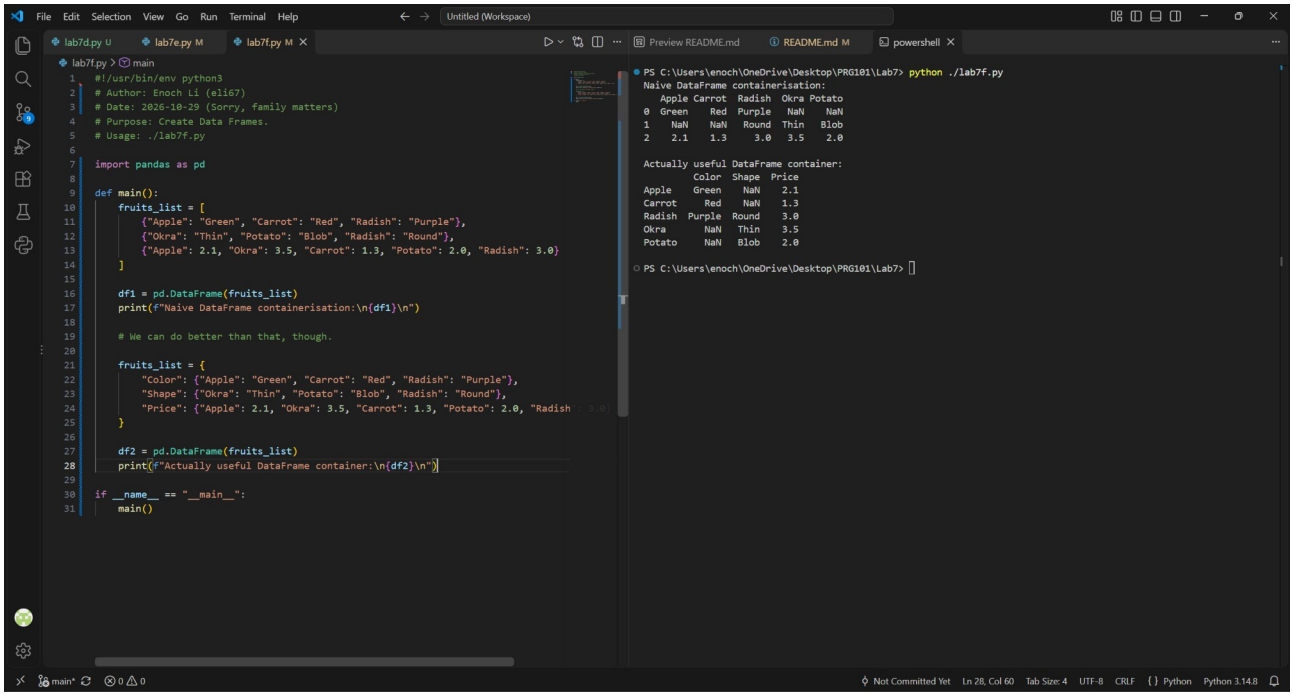
Again, it's not entirely clear what all the instructions refer to since I'm doing this in advance.



I guess I just have to hope that all of these are the intended solutions because there are *other* ways to accomplish this.

Lab7f.py

This is where the vagueness of the instructions really comes to bite me in the arse because all these dictionaries are implied to be nested within a larger dictionary that lists these the attributes of these objects.



Not knowing what to do, I did both. I hope it's what you wanted.

Lab7g.py

So, there's a lot to unpack here. Let's just start with the most obvious: the sheer number of unique genres being combinations of genres:

```
>>> movies['Genre'].unique()
<StringArray>
[                                    'Drama',
                               'Drama | Crime',
           'Drama | Action | Thriller | Crime',
                 'Drama | Biography | History',
        'Drama | Adventure | Fantasy | Action',
                                     'Western',
                 'Drama | Fantasy | Adventure',
                              'Drama | Romance',
                'Adventure | Fantasy | Action',
                'Adventure | Action | Thriller',
 ...
                              'Drama | History',
                                      'Mystery',
 'Thriller | Mystery | Drama | Horror | Crime',
                  'Comedy | Romance | Fantasy',
      'Fantasy | Adventure | Musical | Family',
                  'Drama | Romance | Thriller',
     'Drama | Adventure | Fantasy | Animation',
  'Comedy | Romance | Music | Drama | Musical',
                 'Adventure | Comedy | Action',
  'Comedy | Thriller | Drama | Action | Crime']
Length: 117, dtype: str
>>>
```

That is a lot of genres, but this simply being a CSV table, it takes a bit of "light" processing to find the number of *actually unique* genres:

```
>>> movies['Genre'].str.split('|').explode().str.strip().unique()
<StringArray>
[    'Drama',     'Crime',    'Action',  'Thriller', 'Biography',   'History',
 'Adventure',   'Fantasy',   'Western',   'Romance',   'Mystery',    'Family',
       'War',    'Comedy', 'Animation',    'Horror',     'Music',   'Musical',
     'Sport']
Length: 19, dtype: str
>>>
```

That might look like a lot, but all it took was reading the documentation, a skill I worry might disappear due to the rise in the use of AI. Besides, this is what Unix pipelines are for.

DataFrames are subscriptable, meaning you can browse the data inside them by putting conditions inside index brackets much like how a sub-query works in SQL; this is how you retrieve all of the movies that *aren't* made in the US, with the brackets at the end specifying which columns to show:

```
>>> movies[movies['Origin'] != 'USA'][["Title", "Origin"]]
                                                Title                          Origin
3                                    The Dark Knight                       USA | UK
6          The Lord of the Rings: The Return of the King              USA | New Zealand
8                       Il buono, il brutto, il cattivo    West Germany | Italy | Spain
9                                           Fight Club                   USA | Germany
10     The Lord of the Rings: The Fellowship of the Ring              USA | New Zealand
..                                                   ...                            ...
244                                      La La Land                 Hong Kong | USA
246                             Fanny och Alexander  West Germany | Sweden | France
247                                 Tsubaki Sanjûrô                          Japan
248                              Gangs of Wasseypur                          India
249                                        Drishyam                          India

[127 rows x 2 columns]
>>>
```

However, in listing out the `Origin` column, we run into the exact same problem as `Genre`: the fact that, in real life, many movies are shot outside the US but are ultimately produced and released mainly in the US first. So, to fix that, we need to scan the entire field and exclude all rows that have *any* US involvement at all:

```
>>> movies[~movies['Origin'].str.contains('USA', na=False)][["Title", "Origin"]]
                              Title                          Origin
8     Il buono, il brutto, il cattivo    West Germany | Italy | Spain
18              Shichinin no samurai                          Japan
19                    Cidade de Deus                 Brazil | France
24                   La vita è bella                          Italy
26      Sen to Chihiro no kamikakushi                          Japan
..                                ...                            ...
241           Tenkû no shiro Rapyuta                          Japan
246              Fanny och Alexander  West Germany | Sweden | France
247                  Tsubaki Sanjûrô                          Japan
248               Gangs of Wasseypur                          India
249                         Drishyam                          India

[69 rows x 2 columns]
>>>
```

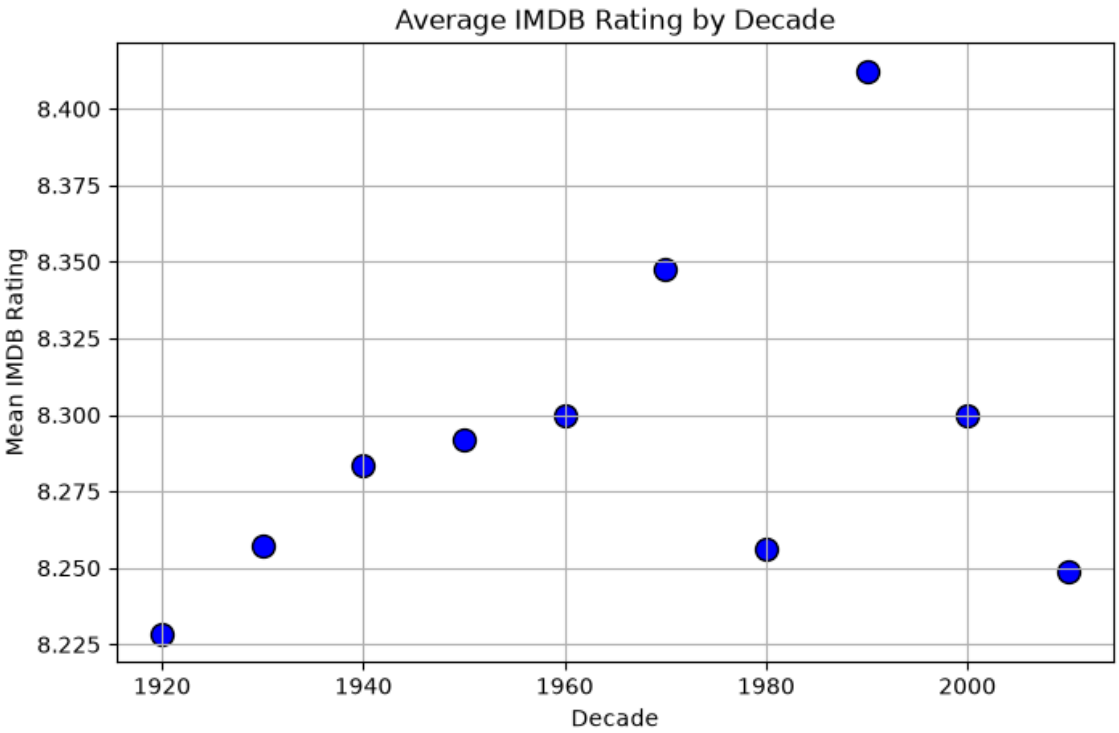
There, the list shrinks to just 69 (nice) movies, which should tell you something about the sheer size of Hollywood. The way this works is by checking whether or not the string 'USA' exists *anywhere* in the field, while the ~ is the logical NOT operator, making it so that it returns only the movies where the string 'USA' is *not* present.

To find the best decade for movies, we need to separate all the movies by decade. To do that, we need to create a new column containing the decades within which each movie was released, and we can do that by defining a new DataFrame that contains all these decades and their average (mean) ratings. Then, we can use `loc` to return the row that has the highest index.

```
>>> groupby_decade = movies.groupby("Decade")['IMDB rating'].mean().reset_index()
>>> best_decade = groupby_decade.loc[groupby_decade['IMDB rating'].idxmax()]
>>> print(f"As it turns out, the best decade for movies is the {int(best_decade['Decade'])}s with an\
 average rating of {best_decade['IMDB rating']:.2f}")
As it turns out, the best decade for movies is the 1990s with an average rating of 8.41
>>>
```

Well, the data speaks for itself. There are probably more elegant solutions, but this is the one I came up with after spending a few hours sifting through all the docs.

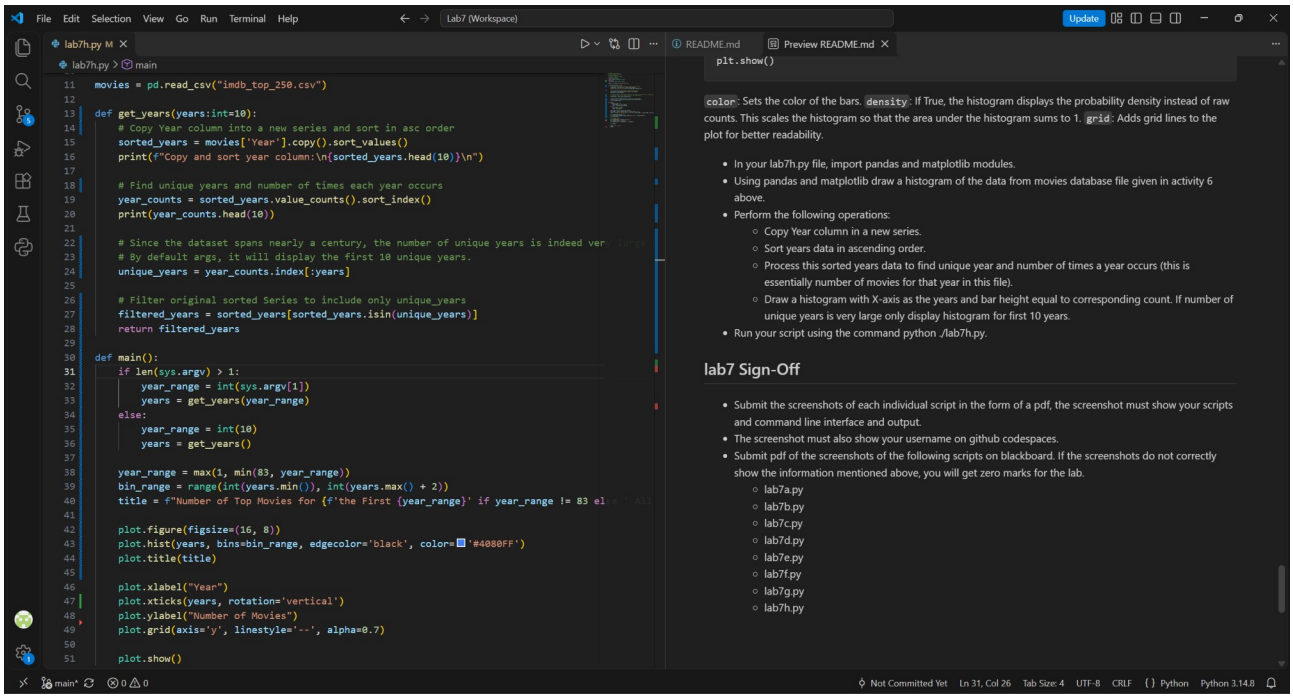
As for the scatterplot, it's `BestMoviesByDecade.png`:



Because the range of the data is so small, the quality spike in quality in the 1990s seems way more dramatic than it really is. Also, it's only a small sample of 250 movies across a hundred years of cinema. Every year, thousands of films are released globally with the US and India contributing the most by volume. This isn't an essay on that, though; it's just a remark on what a tiny sample size the top 250 movies in IMDB truly is. The top 10,000 movies would be a better sample.

Lab7h.py

I spent the better part of Reading Week trying to figure out whether you want me to print every single unique year (of which there are 83) or just the first ten; but after actually running the code, though, 83 is clearly too many even if I were to rotate the labels of the x-axis vertically. However, I still wanted to keep that as an option, so I made the data processing part into its own function and added command-line parsing.



```python
movies = pd.read_csv("imdb_top_250.csv")

def get_years(years:int=10):
    # Copy Year column into a new series and sort in asc order
    sorted_years = movies['Year'].copy().sort_values()
    print(f"Copy and sort year column:\n{sorted_years.head(10)}\n")

    # Find unique years and number of times each year occurs
    year_counts = sorted_years.value_counts().sort_index()
    print(year_counts.head(10))

    # Since the dataset spans nearly a century, the number of unique years is indeed ver
    # By default args, it will display the first 10 unique years.
    unique_years = year_counts.index[:years]

    # Filter original sorted Series to include only unique_years
    filtered_years = sorted_years[sorted_years.isin(unique_years)]
    return filtered_years

def main():
    if len(sys.argv) > 1:
        year_range = int(sys.argv[1])
        years = get_years(year_range)
    else:
        year_range = int(10)
        years = get_years()

    year_range = max(1, min(83, year_range))
    bin_range = range(int(years.min()), int(years.max() + 2))
    title = f"Number of Top Movies for {f'the First {year_range}' if year_range != 83 el

    plot.figure(figsize=(16, 8))
    plot.hist(years, bins=bin_range, edgecolor='black', color='#4080FF')
    plot.title(title)

    plot.xlabel("Year")
    plot.xticks(years, rotation='vertical')
    plot.ylabel("Number of Movies")
    plot.grid(axis='y', linestyle='--', alpha=0.7)

    plot.show()
```

```
plt.show()
```

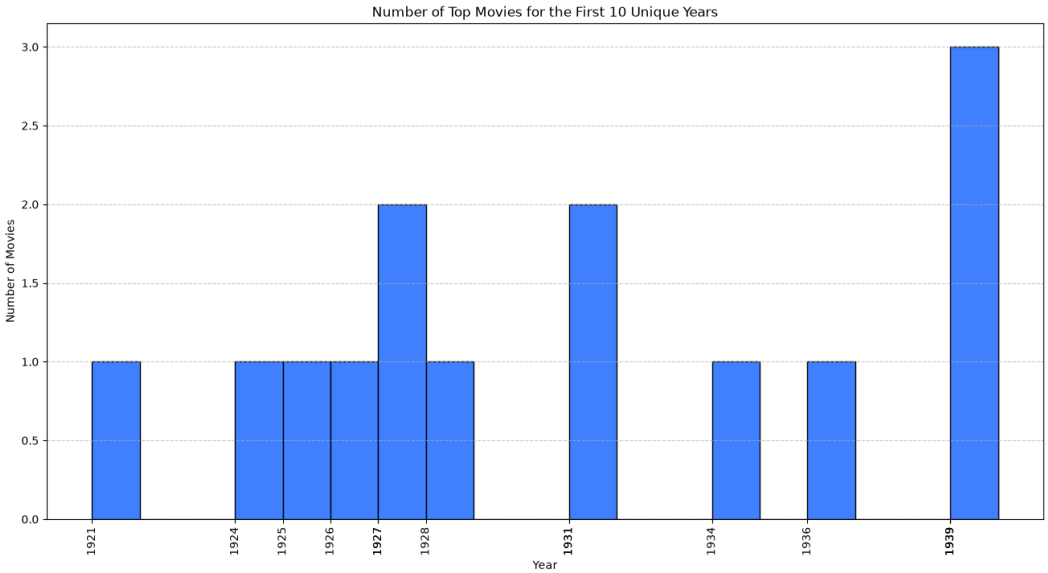
`color`: Sets the color of the bars. `density`: If True, the histogram displays the probability density instead of raw counts. This scales the histogram so that the area under the histogram sums to 1. `grid`: Adds grid lines to the plot for better readability.

- In your lab7h.py file, import pandas and matplotlib modules.
- Using pandas and matplotlib draw a histogram of the data from movies database file given in activity 6 above.
- Perform the following operations:
  - Copy Year column in a new series.
  - Sort years data in ascending order.
  - Process this sorted years data to find unique year and number of times a year occurs (this is essentially number of movies for that year in this file).
  - Draw a histogram with X-axis as the years and bar height equal to corresponding count. If number of unique years is very large only display histogram for first 10 years.
- Run your script using the command python ./lab7h.py.

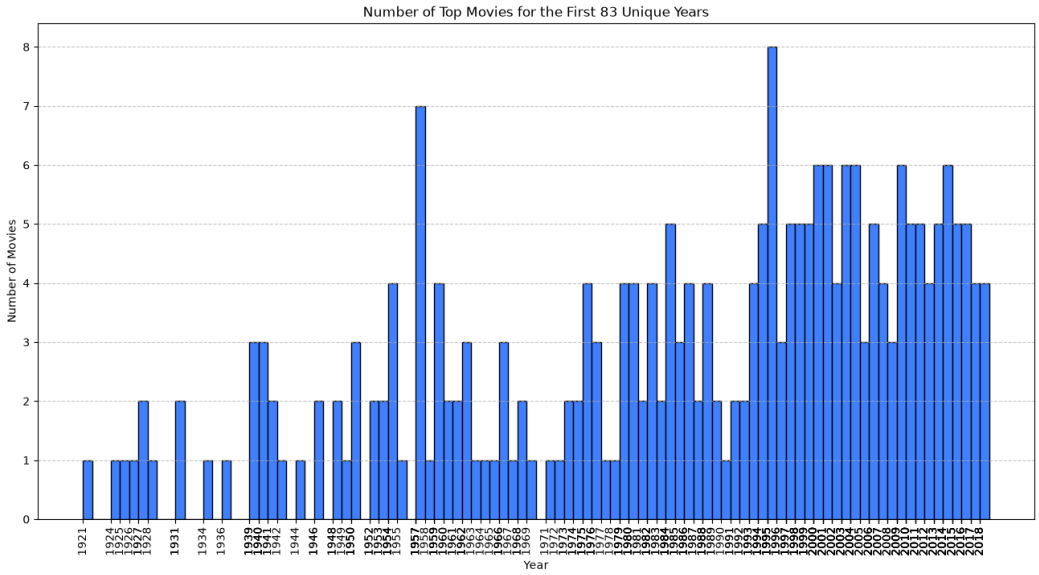
lab7 Sign-Off

- Submit the screenshots of each individual script in the form of a pdf, the screenshot must show your scripts and command line interface and output.
- The screenshot must also show your username on github codespaces.
- Submit pdf of the screenshots of the following scripts on blackboard. If the screenshots do not correctly show the information mentioned above, you will get zero marks for the lab.
  - lab7a.py
  - lab7b.py
  - lab7c.py
  - lab7d.py
  - lab7e.py
  - lab7f.py
  - lab7g.py
  - lab7h.py

Run without arguments, the script defaults to the first ten unique years.



Run with maximum arguments (which programmatically maxes out to 83), it looks like this:



Like I said, the sheer number of almost unreadable. But I did want to find out what what it looked like, so there you go.